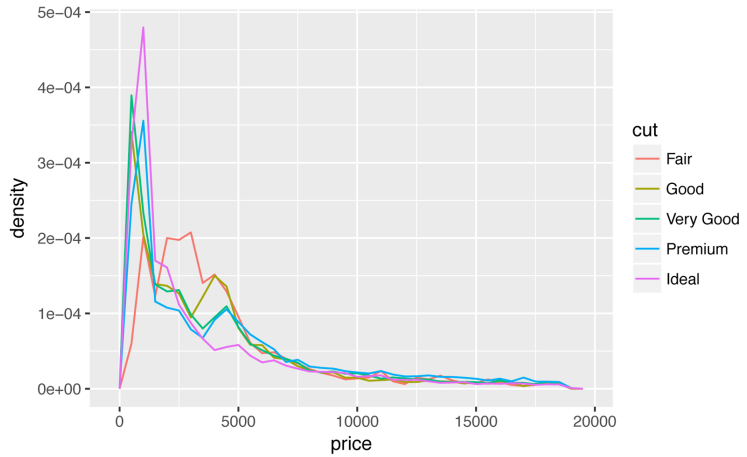
To make the comparison easier we need to swap what is displayed on the y-axis. Instead of displaying count, we'll display *density*, which is the count standardized so that the area under each frequency polygon is one:

```
ggplot(
  data = diamonds,
    mapping = aes(x = price, y = ..density..)
  ) +
   geom_freqpoly(mapping = aes(color = cut), binwidth = 500)
```

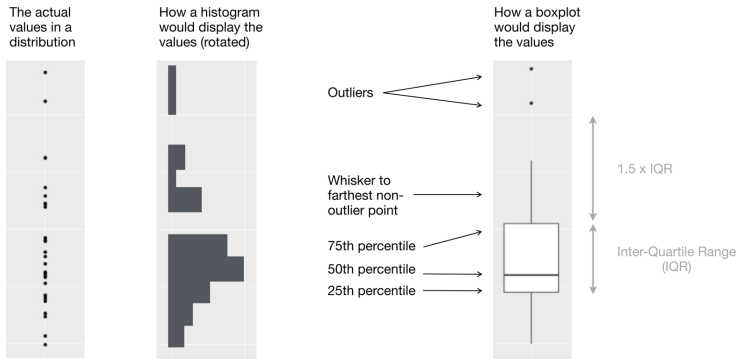


There's something rather surprising about this plot—it appears that fair diamonds (the lowest quality) have the highest average price! But maybe that's because frequency polygons are a little hard to interpret—there's a lot going on in this plot.

Another alternative to display the distribution of a continuous variable broken down by a categorical variable is the boxplot. A *boxplot* is a type of visual shorthand for a distribution of values that is popular among statisticians. Each boxplot consists of:

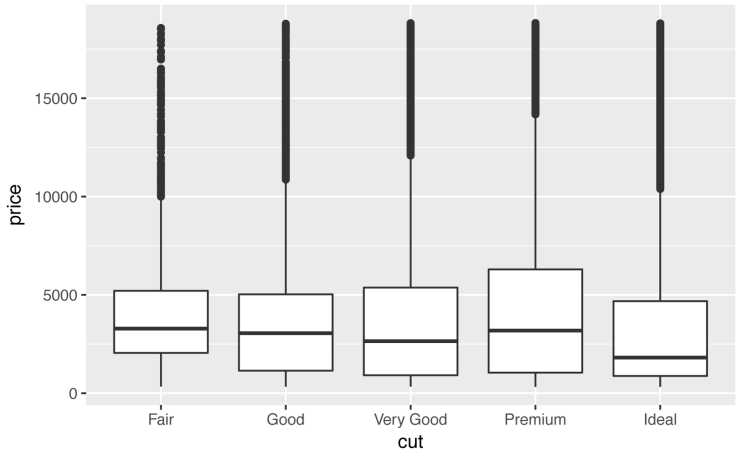
- A box that stretches from the 25th percentile of the distribution to the 75th percentile, a distance known as the interquartile range (IQR). In the middle of the box is a line that displays the median, i.e., 50th percentile, of the distribution. These three lines give you a sense of the spread of the distribution and whether or not the distribution is symmetric about the median or skewed to one side.

- Visual points that display observations that fall more than 1.5 times the IQR from either edge of the box. These outlying points are unusual, so they are plotted individually.
- A line (or whisker) that extends from each end of the box and goes to the farthest nonoutlier point in the distribution.



Let's take a look at the distribution of price by cut using `geom_boxplot()`:

```
ggplot(data = diamonds, mapping = aes(x = cut, y = price)) +
  geom_boxplot()
```



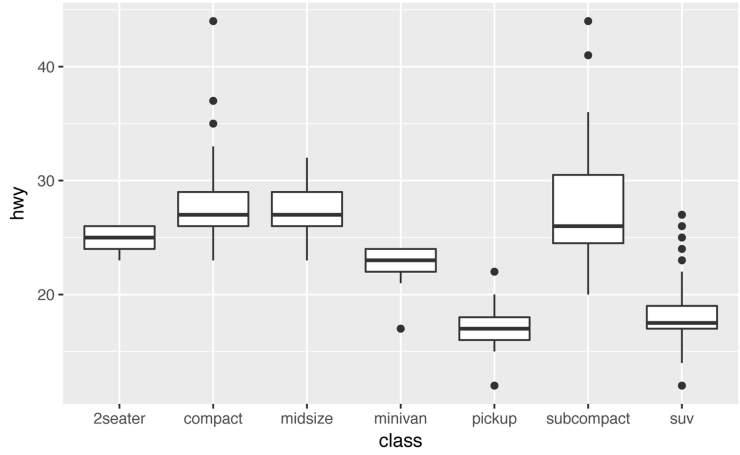
We see much less information about the distribution, but the boxplots are much more compact so we can more easily compare them (and fit more on one plot). It supports the counterintuitive finding

that better quality diamonds are cheaper on average! In the exercises, you'll be challenged to figure out why.

`cut` is an ordered factor: fair is worse than good, which is worse than very good, and so on. Many categorical variables don't have such an intrinsic order, so you might want to reorder them to make a more informative display. One way to do that is with the `reorder()` function.

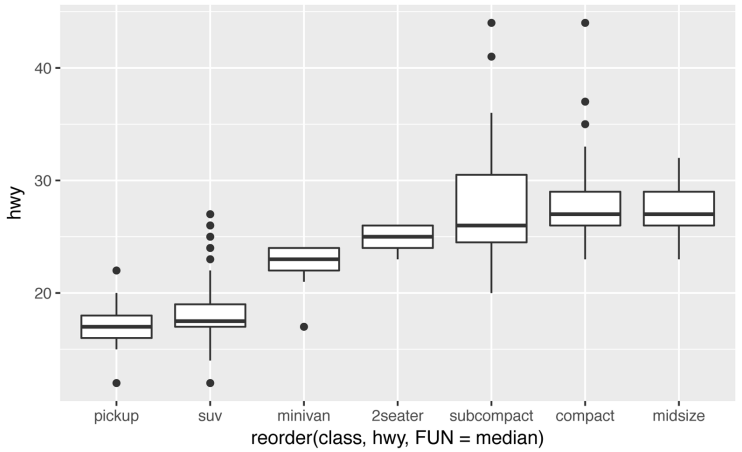
For example, take the `class` variable in the `mpg` dataset. You might be interested to know how highway mileage varies across classes:

```
ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +
  geom_boxplot()
```



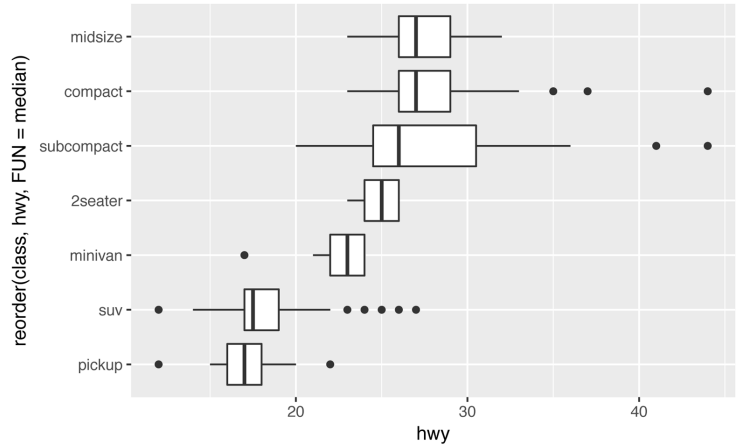
To make the trend easier to see, we can reorder `class` based on the median value of `hwy`:

```
ggplot(data = mpg) +
  geom_boxplot(
    mapping = aes(
      x = reorder(class, hwy, FUN = median),
      y = hwy
    )
  )
```



If you have long variable names, `geom_boxplot()` will work better if you flip it 90°. You can do that with `coord_flip()`:

```
ggplot(data = mpg) +
  geom_boxplot(
    mapping = aes(
      x = reorder(class, hwy, FUN = median),
      y = hwy
    )
  ) +
  coord_flip()
```



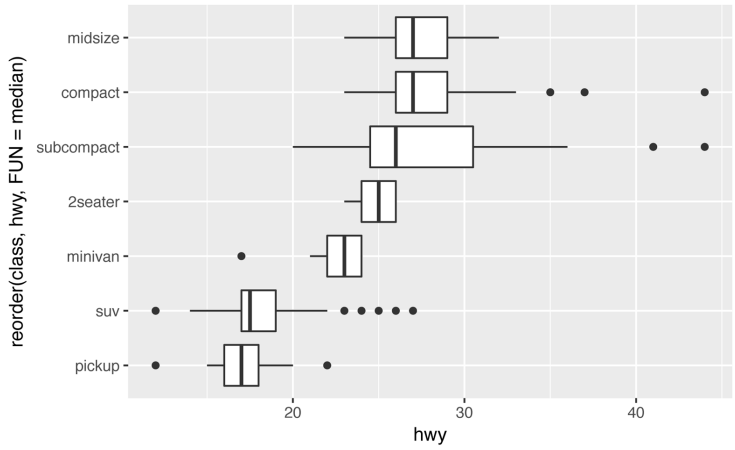
Exercises

1. Use what you've learned to improve the visualization of the departure times of cancelled versus noncancelled flights.
2. What variable in the diamonds dataset is most important for predicting the price of a diamond? How is that variable correlated with cut? Why does the combination of those two relationships lead to lower quality diamonds being more expensive?
3. Install the **ggstance** package, and create a horizontal boxplot. How does this compare to using `coord_flip()`?
4. One problem with boxplots is that they were developed in an era of much smaller datasets and tend to display a prohibitively large number of "outlying values." One approach to remedy this problem is the letter value plot. Install the **lvplot** package, and try using `geom_lv()` to display the distribution of price versus cut. What do you learn? How do you interpret the plots?
5. Compare and contrast `geom_violin()` with a faceted `geom_histogram()`, or a colored `geom_freqpoly()`. What are the pros and cons of each method?
6. If you have a small dataset, it's sometimes useful to use `geom_jitter()` to see the relationship between a continuous and categorical variable. The **ggbeeswarm** package provides a number of methods similar to `geom_jitter()`. List them and briefly describe what each one does.

Two Categorical Variables

To visualize the covariation between categorical variables, you'll need to count the number of observations for each combination. One way to do that is to rely on the built-in `geom_count()`:

```
ggplot(data = diamonds) +
  geom_count(mapping = aes(x = cut, y = color))
```



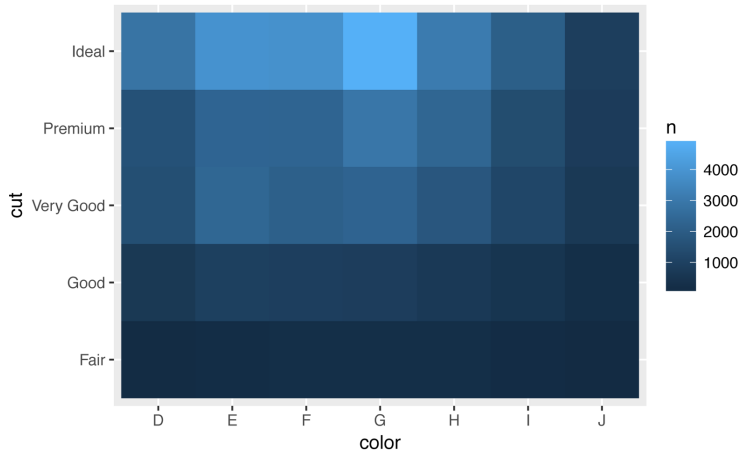
The size of each circle in the plot displays how many observations occurred at each combination of values. Covariation will appear as a strong correlation between specific `x` values and specific `y` values.

Another approach is to compute the count with **dplyr**:

```
diamonds %>%
  count(color, cut)
#> Source: local data frame [35 x 3]
#> Groups: color [?]
#>
#>   color       cut     n
#>   <ord>     <ord> <int>
#> 1     D      Fair   163
#> 2     D      Good   662
#> 3     D Very Good  1513
#> 4     D   Premium  1603
#> 5     D     Ideal  2834
#> 6     E      Fair   224
#> # ... with 29 more rows
```

Then visualize with `geom_tile()` and the fill aesthetic:

```
diamonds %>%
  count(color, cut) %>%
  ggplot(mapping = aes(x = color, y = cut)) +
    geom_tile(mapping = aes(fill = n))
```



If the categorical variables are unordered, you might want to use the **seriation** package to simultaneously reorder the rows and columns in order to more clearly reveal interesting patterns. For larger plots, you might want to try the **d3heatmap** or **heatmaply** packages, which create interactive plots.

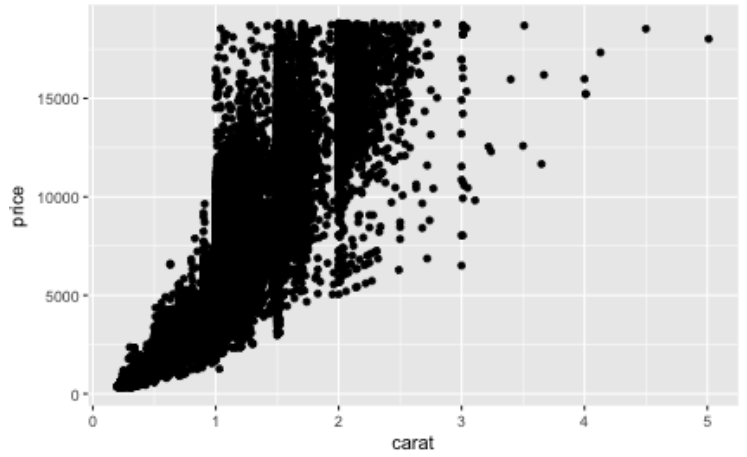
Exercises

1. How could you rescale the `count` dataset to more clearly show the distribution of cut within color, or color within cut?
2. Use `geom_tile()` together with **dplyr** to explore how average flight delays vary by destination and month of year. What makes the plot difficult to read? How could you improve it?
3. Why is it slightly better to use `aes(x = color, y = cut)` rather than `aes(x = cut, y = color)` in the previous example?

Two Continuous Variables

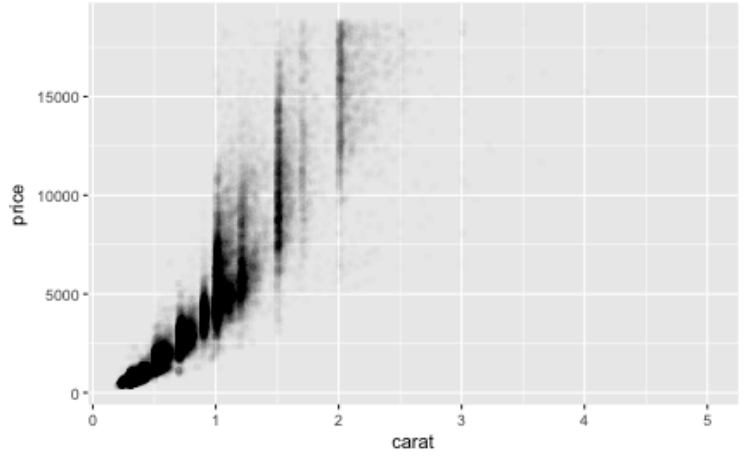
You've already seen one great way to visualize the covariation between two continuous variables: draw a scatterplot with `geom_point()`. You can see covariation as a pattern in the points. For example, you can see an exponential relationship between the carat size and price of a diamond:

```
ggplot(data = diamonds) +
  geom_point(mapping = aes(x = carat, y = price))
```



Scatterplots become less useful as the size of your dataset grows, because points begin to overplot, and pile up into areas of uniform black (as in the preceding scatterplot). You've already seen one way to fix the problem, using the `alpha` aesthetic to add transparency:

```
ggplot(data = diamonds) +
  geom_point(
    mapping = aes(x = carat, y = price),
    alpha = 1 / 100
  )
```



But using transparency can be challenging for very large datasets. Another solution is to use bin. Previously you used `geom_histo`

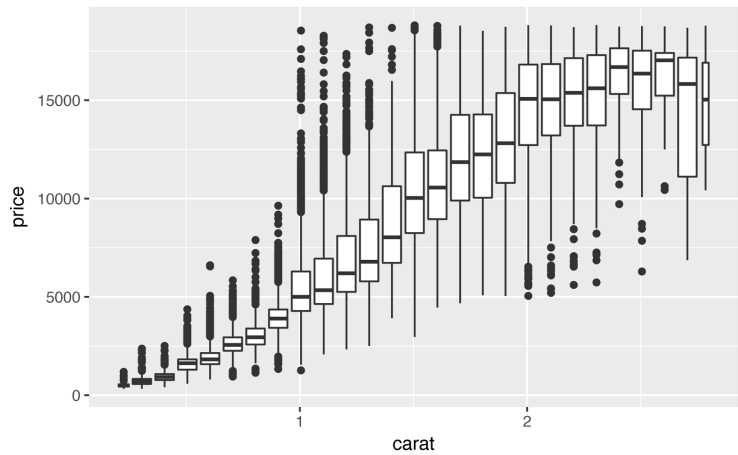
gram() and geom_freqpoly() to bin in one dimension. Now you'll learn how to use geom_bin2d() and geom_hex() to bin in two dimensions.

geom_bin2d() and geom_hex() divide the coordinate plane into 2D bins and then use a fill color to display how many points fall into each bin. geom_bin2d() creates rectangular bins. geom_hex() creates hexagonal bins. You will need to install the **hexbin** package to use geom_hex():

```
ggplot(data = smaller) +
  geom_bin2d(mapping = aes(x = carat, y = price))

# install.packages("hexbin")
ggplot(data = smaller) +
  geom_hex(mapping = aes(x = carat, y = price))
#> Loading required package: methods
```

Another option is to bin one continuous variable so it acts like a categorical variable. Then you can use one of the techniques for visualizing the combination of a categorical and a continuous variable that you learned about. For example, you could bin carat and then for each group, display a boxplot:

```
ggplot(data = smaller, mapping = aes(x = carat, y = price)) +
  geom_boxplot(mapping = aes(group = cut_width(carat, 0.1)))
```

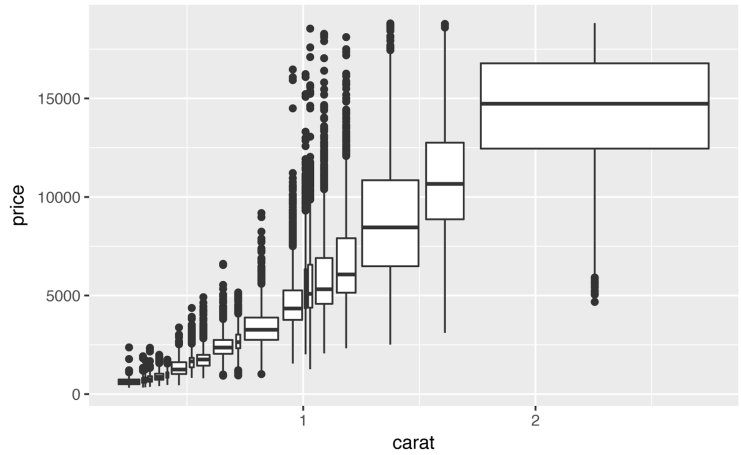


cut_width(x, width), as used here, divides x into bins of width width. By default, boxplots look roughly the same (apart from the number of outliers) regardless of how many observations there are,

so it's difficult to tell that each boxplot summarizes a different number of points. One way to show that is to make the width of the boxplot proportional to the number of points with `varwidth = TRUE`.

Another approach is to display approximately the same number of points in each bin. That's the job of `cut_number()`:

```
ggplot(data = smaller, mapping = aes(x = carat, y = price)) +
  geom_boxplot(mapping = aes(group = cut_number(carat, 20)))
```

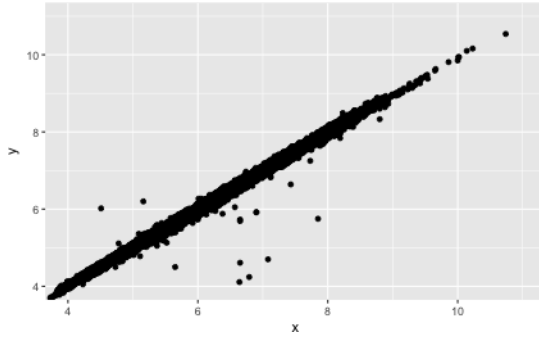


Exercises

1. Instead of summarizing the conditional distribution with a boxplot, you could use a frequency polygon. What do you need to consider when using `cut_width()` versus `cut_number()`? How does that impact a visualization of the 2D distribution of `carat` and `price`?
2. Visualize the distribution of `carat`, partitioned by `price`.
3. How does the price distribution of very large diamonds compare to small diamonds. Is it as you expect, or does it surprise you?
4. Combine two of the techniques you've learned to visualize the combined distribution of `cut`, `carat`, and `price`.
5. Two-dimensional plots reveal outliers that are not visible in one-dimensional plots. For example, some points in the following plot have an unusual combination of `x` and `y` values, which

makes the points outliers even though their x and y values appear normal when examined separately:

```
ggplot(data = diamonds) +
  geom_point(mapping = aes(x = x, y = y)) +
  coord_cartesian(xlim = c(4, 11), ylim = c(4, 11))
```



Why is a scatterplot a better display than a binned plot for this case?

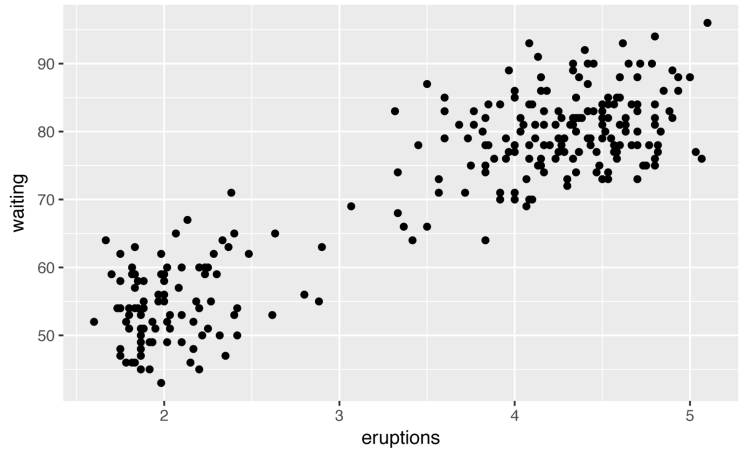
Patterns and Models

Patterns in your data provide clues about relationships. If a systematic relationship exists between two variables it will appear as a pattern in the data. If you spot a pattern, ask yourself:

- Could this pattern be due to coincidence (i.e., random chance)?
- How can you describe the relationship implied by the pattern?
- How strong is the relationship implied by the pattern?
- What other variables might affect the relationship?
- Does the relationship change if you look at individual subgroups of the data?

A scatterplot of Old Faithful eruption lengths versus the wait time between eruptions shows a pattern: longer wait times are associated with longer eruptions. The scatterplot also displays the two clusters that we noticed earlier:

```
ggplot(data = faithful) +
  geom_point(mapping = aes(x = eruptions, y = waiting))
```

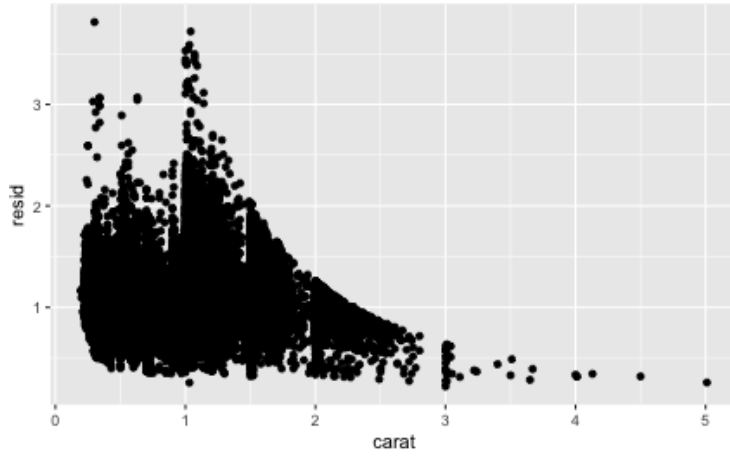


Patterns provide one of the most useful tools for data scientists because they reveal covariation. If you think of variation as a phenomenon that creates uncertainty, covariation is a phenomenon that reduces it. If two variables covary, you can use the values of one variable to make better predictions about the values of the second. If the covariation is due to a causal relationship (a special case), then you can use the value of one variable to control the value of the second.

Models are a tool for extracting patterns out of data. For example, consider the diamonds data. It's hard to understand the relationship between cut and price, because cut and carat, and carat and price, are tightly related. It's possible to use a model to remove the very strong relationship between price and carat so we can explore the subtleties that remain. The following code fits a model that predicts `price` from `carat` and then computes the residuals (the difference between the predicted value and the actual value). The residuals give us a view of the price of the diamond, once the effect of carat has been removed:

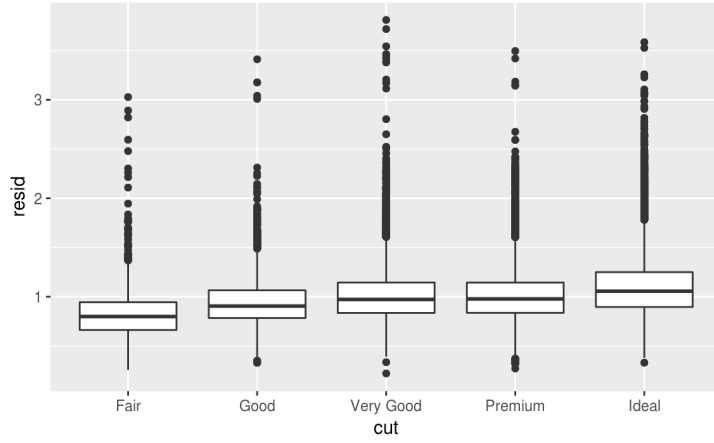
```
library(modelr)

mod <- lm(log(price) ~ log(carat), data = diamonds)

diamonds2 <- diamonds %>%
  add_residuals(mod) %>%
  mutate(resid = exp(resid))
```

```
ggplot(data = diamonds2) +
  geom_point(mapping = aes(x = carat, y = resid))
```



Once you've removed the strong relationship between carat and price, you can see what you expect in the relationship between cut and price—relative to their size, better quality diamonds are more expensive:

```
ggplot(data = diamonds2) +
  geom_boxplot(mapping = aes(x = cut, y = resid))
```



You'll learn how models, and the **modelr** package, work in the final part of the book, Part IV. We're saving modeling for later because

understanding what models are and how they work is easiest once you have the tools of data wrangling and programming in hand.

ggplot2 Calls

As we move on from these introductory chapters, we'll transition to a more concise expression of **ggplot2** code. So far we've been very explicit, which is helpful when you are learning:

```
ggplot(data = faithful, mapping = aes(x = eruptions)) +
  geom_freqpoly(binwidth = 0.25)
```

Typically, the first one or two arguments to a function are so important that you should know them by heart. The first two arguments to `ggplot()` are `data` and `mapping`, and the first two arguments to `aes()` are `x` and `y`. In the remainder of the book, we won't supply those names. That saves typing, and, by reducing the amount of boilerplate, makes it easier to see what's different between plots. That's a really important programming concern that we'll come back to in Chapter 15.

Rewriting the previous plot more concisely yields:

```
ggplot(faithful, aes(eruptions)) +
  geom_freqpoly(binwidth = 0.25)
```

Sometimes we'll turn the end of a pipeline of data transformation into a plot. Watch for the transition from `%>%` to `+`. I wish this transition wasn't necessary but unfortunately **ggplot2** was created before the pipe was discovered:

```
diamonds %>%
  count(cut, clarity) %>%
  ggplot(aes(clarity, cut, fill = n)) +
    geom_tile()
```

Learning More

If you want learn more about the mechanics of **ggplot2**, I'd highly recommend grabbing a copy of the **ggplot2** book. It's been recently updated, so it includes **dplyr** and **tidyr** code, and has much more space to explore all the facets of visualization. Unfortunately the book isn't generally available for free, but if you have a connection to a university you can probably get an electronic version for free through SpringerLink.

Another useful resource is the *R Graphics Cookbook* by Winston Chang. Much of the contents are available online at *http://www.cookbook-r.com/Graphs/*.

I also recommend *Graphical Data Analysis with R*, by Antony Unwin. This is a book-length treatment similar to the material covered in this chapter, but has the space to go into much greater depth.

CHAPTER 6

Workflow: Projects

One day you will need to quit R, go do something else, and return to your analysis the next day. One day you will be working on multiple analyses simultaneously that all use R and you want to keep them separate. One day you will need to bring data from the outside world into R and send numerical results and figures from R back out into the world. To handle these real-life situations, you need to make two decisions:

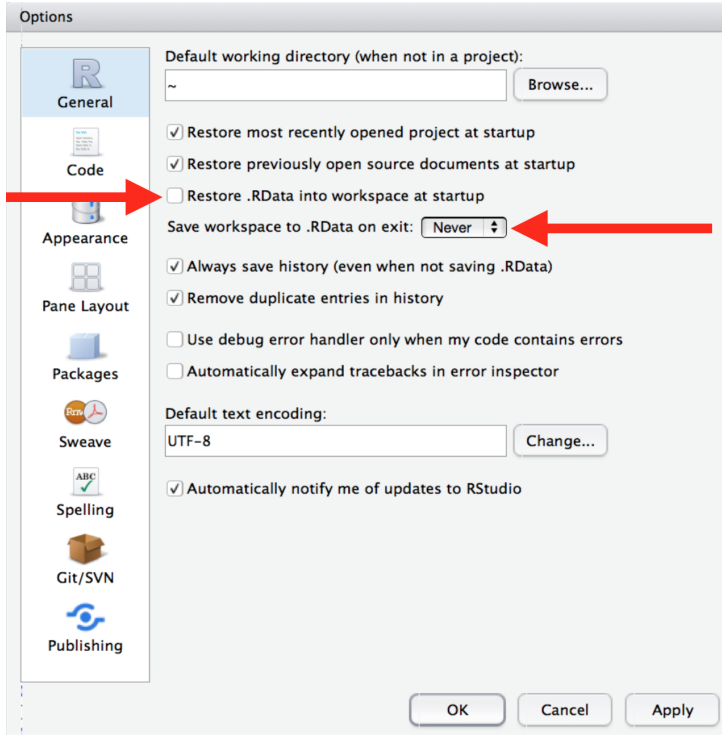
1. What about your analysis is "real," i.e., what will you save as your lasting record of what happened?
2. Where does your analysis "live"?

What Is Real?

As a beginning R user, it's OK to consider your environment (i.e., the objects listed in the environment pane) "real." However, in the long run, you'll be much better off if you consider your R scripts as "real."

With your R scripts (and your data files), you can re-create the environment. It's much harder to re-create your R scripts from your environment! You'll either have to retype a lot of code from memory (making mistakes all the way) or you'll have to carefully mine your R history.

To foster this behavior, I highly recommend that you instruct RStudio not to preserve your workspace between sessions:



This will cause you some short-term pain, because now when you restart RStudio it will not remember the results of the code that you ran last time. But this short-term pain will save you long-term agony because it forces you to capture all important interactions in your code. There's nothing worse than discovering three months after the fact that you've only stored the results of an important calculation in your workspace, not the calculation itself in your code.

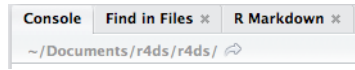
There is a great pair of keyboard shortcuts that will work together to make sure you've captured the important parts of your code in the editor:

- Press Cmd/Ctrl-Shift-F10 to restart RStudio.
- Press Cmd/Ctrl-Shift-S to rerun the current script.

I use this pattern hundreds of times a week.

Where Does Your Analysis Live?

R has a powerful notion of the *working directory*. This is where R looks for files that you ask it to load, and where it will put any files that you ask it to save. RStudio shows your current working directory at the top of the console:



And you can print this out in R code by running `getwd()`:

```
getwd()
#> [1] "/Users/hadley/Documents/r4ds/r4ds"
```

As a beginning R user, it's OK to let your home directory, documents directory, or any other weird directory on your computer be R's working directory. But you're six chapters into this book, and you're no longer a rank beginner. Very soon now you should evolve to organizing your analytical projects into directories and, when working on a project, setting R's working directory to the associated directory.

I do not recommend it, but you can also set the working directory from within R:

```
setwd("/path/to/my/CoolProject")
```

But you should never do this because there's a better way; a way that also puts you on the path to managing your R work like an expert.

Paths and Directories

Paths and directories are a little complicated because there are two basic styles of paths: Mac/Linux and Windows. There are three chief ways in which they differ:

- The most important difference is how you separate the components of the path. Mac and Linux use slashes (e.g., `plots/diamonds.pdf`) and Windows uses backslashes (e.g., `plots\diamonds.pdf`). R can work with either type (no matter what platform you're currently using), but unfortunately, backslashes mean something special to R, and to get a single backslash in the path, you need to type two backslashes! That makes life frus-

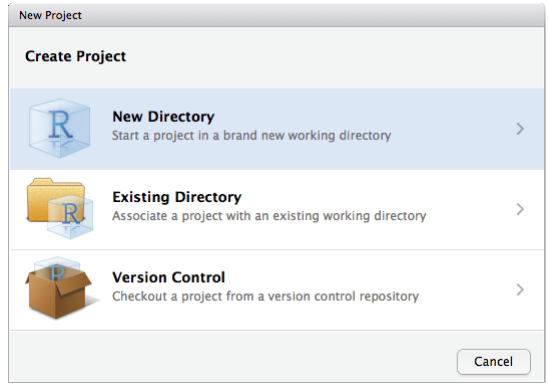
trating, so I recommend always using the Linux/Max style with forward slashes.

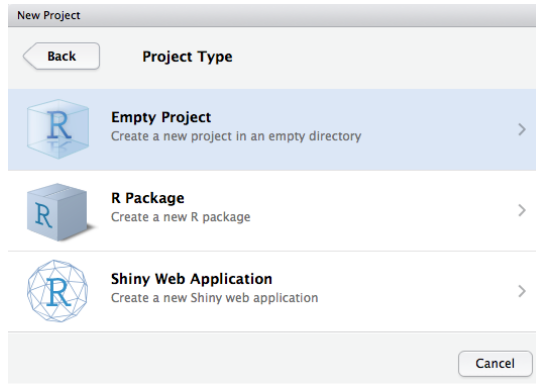
- Absolute paths (i.e., paths that point to the same place regardless of your working directory) look different. In Windows they start with a drive letter (e.g., `C:`) or two backslashes (e.g., `\\servername`) and in Mac/Linux they start with a slash "/" (e.g., `/users/hadley`). You should *never* use absolute paths in your scripts, because they hinder sharing: no one else will have exactly the same directory configuration as you.
- The last minor difference is the place that ~ points to. ~ is a convenient shortcut to your home directory. Windows doesn't really have the notion of a home directory, so it instead points to your documents directory.

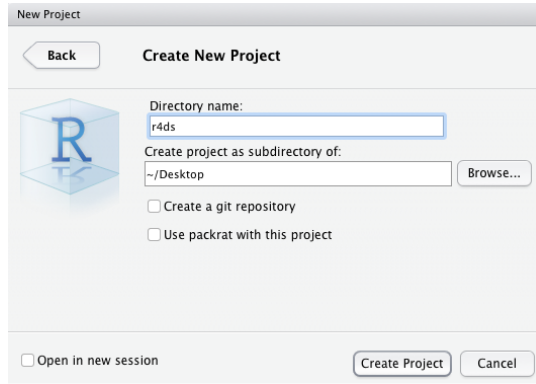
RStudio Projects

R experts keep all the files associated with a project together—input data, R scripts, analytical results, figures. This is such a wise and common practice that RStudio has built-in support for this via *projects*.

Let's make a project for you to use while you're working through the rest of this book. Click File → New Project, then:







Call your project `r4ds` and think carefully about which *subdirectory* you put the project in. If you don't store it somewhere sensible, it will be hard to find it in the future!

Once this process is complete, you'll get a new RStudio project just for this book. Check that the "home" directory of your project is the current working directory:

```
getwd()
#> [1] /Users/hadley/Documents/r4ds/r4ds
```

Whenever you refer to a file with a relative path it will look for it here.

Now enter the following commands in the script editor, and save the file, calling it *diamonds.R*. Next, run the complete script, which will

save a PDF and CSV file into your project directory. Don't worry about the details, you'll learn them later in the book:

```
library(tidyverse)

ggplot(diamonds, aes(carat, price)) +
  geom_hex()
ggsave("diamonds.pdf")

write_csv(diamonds, "diamonds.csv")
```

Quit RStudio. Inspect the folder associated with your project—notice the *.Rproj* file. Double-click that file to reopen the project. Notice you get back to where you left off: it's the same working directory and command history, and all the files you were working on are still open. Because you followed my instructions above, you will, however, have a completely fresh environment, guaranteeing that you're starting with a clean slate.

In your favorite OS-specific way, search your computer for *diamonds.pdf* and you will find the PDF (no surprise) but *also the script that created it* (*diamonds.r*). This is huge win! One day you will want to remake a figure or just understand where it came from. If you rigorously save figures to files *with R code* and never with the mouse or the clipboard, you will be able to reproduce old work with ease!

Summary

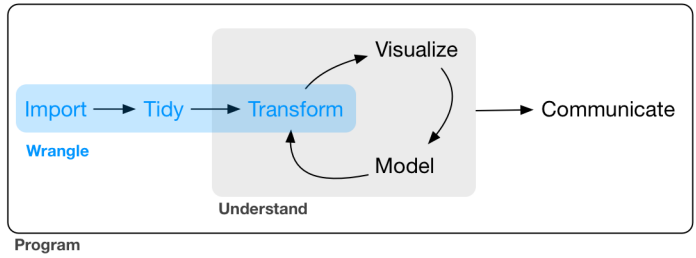
In summary, RStudio projects give you a solid workflow that will serve you well in the future:

- Create an RStudio project for each data analyis project.
- Keep data files there; we'll talk about loading them into R in Chapter 8.
- Keep scripts there; edit them, and run them in bits or as a whole.
- Save your outputs (plots and cleaned data) there.
- Only ever use relative paths, not absolute paths.

Everything you need is in one place, and cleanly separated from all the other projects that you are working on.

PART II
Wrangle

In this part of the book, you'll learn about data wrangling, the art of getting your data into R in a useful form for visualization and modeling. Data wrangling is very important: without it you can't work with your own data! There are three main parts to data wrangling:



This part of the book proceeds as follows:

- In Chapter 7, you'll learn about the variant of the data frame that we use in this book: the *tibble*. You'll learn what makes them different from regular data frames, and how you can construct them "by hand."
- In Chapter 8, you'll learn how to get your data from disk and into R. We'll focus on plain-text rectangular formats, but will give you pointers to packages that help with other types of data.

- In Chapter 9, you'll learn about tidy data, a consistent way of storing your data that makes transformation, visualization, and modeling easier. You'll learn the underlying principles, and how to get your data into a tidy form.

Data wrangling also encompasses data transformation, which you've already learned a little about. Now we'll focus on new skills for three specific types of data you will frequently encounter in practice:

- Chapter 10 will give you tools for working with multiple interrelated datasets.
- Chapter 11 will introduce regular expressions, a powerful tool for manipulating strings.
- Chapter 12 will show you how R stores categorical data. They are used when a variable has a fixed set of possible values, or when you want to use a nonalphabetical ordering of a string.
- Chapter 13 will give you the key tools for working with dates and date-times.

CHAPTER 7

Tibbles with tibble

Introduction

Throughout this book we work with "tibbles" instead of R's traditional `data.frame`. Tibbles *are* data frames, but they tweak some older behaviors to make life a little easier. R is an old language, and some things that were useful 10 or 20 years ago now get in your way. It's difficult to change base R without breaking existing code, so most innovation occurs in packages. Here we will describe the *tibble* package, which provides opinionated data frames that make working in the tidyverse a little easier. In most places, I'll use the terms tibble and data frame interchangeably; when I want to draw particular attention to R's built-in data frame, I'll call them `data.frame`s.

If this chapter leaves you wanting to learn more about tibbles, you might enjoy `vignette("tibble")`.

Prerequisites

In this chapter we'll explore the **tibble** package, part of the core tidyverse.

```
library(tidyverse)
```

Creating Tibbles

Almost all of the functions that you'll use in this book produce tibbles, as tibbles are one of the unifying features of the tidyverse. Most

other R packages use regular data frames, so you might want to coerce a data frame to a tibble. You can do that with `as_tibble()`:

```
as_tibble(iris)
#> # A tibble: 150 × 5
#>   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
#>          <dbl>       <dbl>        <dbl>       <dbl>  <fctr>
#> 1          5.1         3.5          1.4         0.2  setosa
#> 2          4.9         3.0          1.4         0.2  setosa
#> 3          4.7         3.2          1.3         0.2  setosa
#> 4          4.6         3.1          1.5         0.2  setosa
#> 5          5.0         3.6          1.4         0.2  setosa
#> 6          5.4         3.9          1.7         0.4  setosa
#> # ... with 144 more rows
```

You can create a new tibble from individual vectors with `tibble()`. `tibble()` will automatically recycle inputs of length 1, and allows you to refer to variables that you just created, as shown here:

```
tibble(
  x = 1:5,
  y = 1,
  z = x ^ 2 + y
)
#> # A tibble: 5 × 3
#>       x     y     z
#>   <int> <dbl> <dbl>
#> 1     1     1     2
#> 2     2     1     5
#> 3     3     1    10
#> 4     4     1    17
#> 5     5     1    26
```

If you're already familiar with `data.frame()`, note that `tibble()` does much less: it never changes the type of the inputs (e.g., it never converts strings to factors!), it never changes the names of variables, and it never creates row names.

It's possible for a tibble to have column names that are not valid R variable names, aka *nonsyntactic* names. For example, they might not start with a letter, or they might contain unusual characters like a space. To refer to these variables, you need to surround them with backticks, `` ` ``:

```
tb <- tibble(
  `:)` = "smile",
  ` ` = "space",
  `2000` = "number"
)
tb
```

```
#> # A tibble: 1 × 3
#>    `:)`   ` ` `2000`
#>   <chr> <chr>  <chr>
#> 1 smile space number
```

You'll also need the backticks when working with these variables in other packages, like **ggplot2**, **dplyr**, and **tidyr**.

Another way to create a tibble is with `tribble()`, short for *transposed tibble*. `tribble()` is customized for data entry in code: column headings are defined by formulas (i.e., they start with ~), and entries are separated by commas. This makes it possible to lay out small amounts of data in easy-to-read form:

```
tribble(
  ~x, ~y, ~z,
  #--|--|----
  "a", 2, 3.6,
  "b", 1, 8.5
)
#> # A tibble: 2 × 3
#>       x     y     z
#>   <chr> <dbl> <dbl>
#> 1     a     2   3.6
#> 2     b     1   8.5
```

I often add a comment (the line starting with #) to make it really clear where the header is.

Tibbles Versus data.frame

There are two main differences in the usage of a tibble versus a classic `data.frame`: printing and subsetting.

Printing

Tibbles have a refined print method that shows only the first 10 rows, and all the columns that fit on screen. This makes it much easier to work with large data. In addition to its name, each column reports its type, a nice feature borrowed from `str()`:

```
tibble(
  a = lubridate::now() + runif(1e3) * 86400,
  b = lubridate::today() + runif(1e3) * 30,
  c = 1:1e3,
  d = runif(1e3),
  e = sample(letters, 1e3, replace = TRUE)
)
```

```
#> # A tibble: 1,000 × 5
#>                      a          b     c     d     e
#>                 <dttm>     <date> <int> <dbl> <chr>
#> 1 2016-10-10 17:14:14 2016-10-17     1 0.368     h
#> 2 2016-10-11 11:19:24 2016-10-22     2 0.612     n
#> 3 2016-10-11 05:43:03 2016-11-01     3 0.415     l
#> 4 2016-10-10 19:04:20 2016-10-31     4 0.212     x
#> 5 2016-10-10 15:28:37 2016-10-28     5 0.733     a
#> 6 2016-10-11 02:29:34 2016-10-24     6 0.460     v
#> # ... with 994 more rows
```

Tibbles are designed so that you don't accidentally overwhelm your console when you print large data frames. But sometimes you need more output than the default display. There are a few options that can help.

First, you can explicitly `print()` the data frame and control the number of rows (`n`) and the `width` of the display. `width = Inf` will display all columns:

```
nycflights13::flights %>%
  print(n = 10, width = Inf)
```

You can also control the default print behavior by setting options:

- `options(tibble.print_max = n, tibble.print_min = m)`: if more than `m` rows, print only `n` rows. Use `options(dplyr.print_min = Inf)` to always show all rows.
- Use `options(tibble.width = Inf)` to always print all columns, regardless of the width of the screen.

You can see a complete list of options by looking at the package help with `package?tibble`.

A final option is to use RStudio's built-in data viewer to get a scrollable view of the complete dataset. This is also often useful at the end of a long chain of manipulations:

```
nycflights13::flights %>%
  View()
```

Subsetting

So far all the tools you've learned have worked with complete data frames. If you want to pull out a single variable, you need some new tools, `$` and `[[`. `[[` can extract by name or position; `$` only extracts by name but is a little less typing:

```
df <- tibble(
  x = runif(5),
  y = rnorm(5)
)

# Extract by name
df$x
#> [1] 0.434 0.395 0.548 0.762 0.254
df[["x"]]
#> [1] 0.434 0.395 0.548 0.762 0.254

# Extract by position
df[[1]]
#> [1] 0.434 0.395 0.548 0.762 0.254
```

To use these in a pipe, you'll need to use the special placeholder `.`:

```
df %>% .$x
#> [1] 0.434 0.395 0.548 0.762 0.254
df %>% .[["x"]]
#> [1] 0.434 0.395 0.548 0.762 0.254
```

Compared to a `data.frame`, tibbles are more strict: they never do partial matching, and they will generate a warning if the column you are trying to access does not exist.

Interacting with Older Code

Some older functions don't work with tibbles. If you encounter one of these functions, use `as.data.frame()` to turn a tibble back to a `data.frame`:

```
class(as.data.frame(tb))
#> [1] "data.frame"
```

The main reason that some older functions don't work with tibbles is the `[` function. We don't use `[` much in this book because `dplyr::filter()` and `dplyr::select()` allow you to solve the same problems with clearer code (but you will learn a little about it in "Subsetting" on page 300). With base R data frames, `[` sometimes returns a data frame, and sometimes returns a vector. With tibbles, `[` always returns another tibble.

Exercises

1. How can you tell if an object is a tibble? (Hint: try printing `mtcars`, which is a regular data frame.)

2. Compare and contrast the following operations on a `data.frame` and equivalent tibble. What is different? Why might the default data frame behaviors cause you frustration?

```
df <- data.frame(abc = 1, xyz = "a")
df$x
df[, "xyz"]
df[, c("abc", "xyz")]
```

3. If you have the name of a variable stored in an object, e.g., `var <- "mpg"`, how can you extract the reference variable from a tibble?
4. Practice referring to nonsyntactic names in the following data frame by:
   a. Extracting the variable called `1`.
   b. Plotting a scatterplot of `1` versus `2`.
   c. Creating a new column called `3`, which is `2` divided by `1`.
   d. Renaming the columns to `one`, `two`, and `three`:

```
annoying <- tibble(
  `1` = 1:10,
  `2` = `1` * 2 + rnorm(length(`1`))
)
```

5. What does `tibble::enframe()` do? When might you use it?
6. What option controls how many additional column names are printed at the footer of a tibble?

CHAPTER 8

Data Import with readr

Introduction

Working with data provided by R packages is a great way to learn the tools of data science, but at some point you want to stop learning and start working with your own data. In this chapter, you'll learn how to read plain-text rectangular files into R. Here, we'll only scratch the surface of data import, but many of the principles will translate to other forms of data. We'll finish with a few pointers to packages that are useful for other types of data.

Prerequisites

In this chapter, you'll learn how to load flat files in R with the **readr** package, which is part of the core tidyverse.

```
library(tidyverse)
```

Getting Started

Most of **readr**'s functions are concerned with turning flat files into data frames:

- `read_csv()` reads comma-delimited files, `read_csv2()` reads semicolon-separated files (common in countries where `,` is used as the decimal place), `read_tsv()` reads tab-delimited files, and `read_delim()` reads in files with any delimiter.

- `read_fwf()` reads fixed-width files. You can specify fields either by their widths with `fwf_widths()` or their position with `fwf_positions()`. `read_table()` reads a common variation of fixed-width files where columns are separated by white space.
- `read_log()` reads Apache style log files. (But also check out **webreadr**, which is built on top of `read_log()` and provides many more helpful tools.)

These functions all have similar syntax: once you've mastered one, you can use the others with ease. For the rest of this chapter we'll focus on `read_csv()`. Not only are CSV files one of the most common forms of data storage, but once you understand `read_csv()`, you can easily apply your knowledge to all the other functions in **readr**.

The first argument to `read_csv()` is the most important; it's the path to the file to read:

```
heights <- read_csv("data/heights.csv")
#> Parsed with column specification:
#> cols(
#>   earn = col_double(),
#>   height = col_double(),
#>   sex = col_character(),
#>   ed = col_integer(),
#>   age = col_integer(),
#>   race = col_character()
#> )
```

When you run `read_csv()` it prints out a column specification that gives the name and type of each column. That's an important part of **readr**, which we'll come back to in "Parsing a File" on page 137.

You can also supply an inline CSV file. This is useful for experimenting with **readr** and for creating reproducible examples to share with others:

```
read_csv("a,b,c
1,2,3
4,5,6")
#> # A tibble: 2 × 3
#>       a     b     c
#>   <int> <int> <int>
#> 1     1     2     3
#> 2     4     5     6
```

In both cases `read_csv()` uses the first line of the data for the column names, which is a very common convention. There are two cases where you might want to tweak this behavior:

- Sometimes there are a few lines of metadata at the top of the file. You can use `skip = n` to skip the first `n` lines; or use `comment = "#"` to drop all lines that start with (e.g.) `#`:

```
read_csv("The first line of metadata
  The second line of metadata
  x,y,z
  1,2,3", skip = 2)
#> # A tibble: 1 × 3
#>       x     y     z
#>   <int> <int> <int>
#> 1     1     2     3

read_csv("# A comment I want to skip
  x,y,z
  1,2,3", comment = "#")
#> # A tibble: 1 × 3
#>       x     y     z
#>   <int> <int> <int>
#> 1     1     2     3
```

- The data might not have column names. You can use `col_names = FALSE` to tell `read_csv()` not to treat the first row as headings, and instead label them sequentially from `X1` to `Xn`:

```
read_csv("1,2,3\n4,5,6", col_names = FALSE)
#> # A tibble: 2 × 3
#>      X1    X2    X3
#>   <int> <int> <int>
#> 1     1     2     3
#> 2     4     5     6
```

  (`"\n"` is a convenient shortcut for adding a new line. You'll learn more about it and other types of string escape in "String Basics" on page 195.)

  Alternatively you can pass `col_names` a character vector, which will be used as the column names:

```
read_csv("1,2,3\n4,5,6", col_names = c("x", "y", "z"))
#> # A tibble: 2 × 3
#>       x     y     z
#>   <int> <int> <int>
```

```
#> 1     1     2     3
#> 2     4     5     6
```

Another option that commonly needs tweaking is `na`. This specifies the value (or values) that are used to represent missing values in your file:

```
read_csv("a,b,c\n1,2,.", na = ".")
#> # A tibble: 1 × 3
#>       a     b     c
#>   <int> <int> <chr>
#> 1     1     2  <NA>
```

This is all you need to know to read ~75% of CSV files that you'll encounter in practice. You can also easily adapt what you've learned to read tab-separated files with `read_tsv()` and fixed-width files with `read_fwf()`. To read in more challenging files, you'll need to learn more about how **readr** parses each column, turning them into R vectors.

Compared to Base R

If you've used R before, you might wonder why we're not using `read.csv()`. There are a few good reasons to favor **readr** functions over the base equivalents:

- They are typically much faster (~10x) than their base equivalents. Long-running jobs have a progress bar, so you can see what's happening. If you're looking for raw speed, try `data.table::fread()`. It doesn't fit quite so well into the tidyverse, but it can be quite a bit faster.
- They produce tibbles, and they don't convert character vectors to factors, use row names, or munge the column names. These are common sources of frustration with the base R functions.
- They are more reproducible. Base R functions inherit some behavior from your operating system and environment variables, so import code that works on your computer might not work on someone else's.

Exercises

1. What function would you use to read a file where fields are separated with "|"?

2. Apart from `file`, `skip`, and `comment`, what other arguments do `read_csv()` and `read_tsv()` have in common?
3. What are the most important arguments to `read_fwf()`?
4. Sometimes strings in a CSV file contain commas. To prevent them from causing problems they need to be surrounded by a quoting character, like `"` or `'`. By convention, `read_csv()` assumes that the quoting character will be `"`, and if you want to change it you'll need to use `read_delim()` instead. What arguments do you need to specify to read the following text into a data frame?

   ```
   "x,y\n1,'a,b'"
   ```

5. Identify what is wrong with each of the following inline CSV files. What happens when you run the code?

   ```
   read_csv("a,b\n1,2,3\n4,5,6")
   read_csv("a,b,c\n1,2\n1,2,3,4")
   read_csv("a,b\n\"1")
   read_csv("a,b\n1,2\na,b")
   read_csv("a;b\n1;3")
   ```

Parsing a Vector

Before we get into the details of how **readr** reads files from disk, we need to take a little detour to talk about the `parse_*()` functions. These functions take a character vector and return a more specialized vector like a logical, integer, or date:

```
str(parse_logical(c("TRUE", "FALSE", "NA")))
#>  logi [1:3] TRUE FALSE NA
str(parse_integer(c("1", "2", "3")))
#>  int [1:3] 1 2 3
str(parse_date(c("2010-01-01", "1979-10-14")))
#>  Date[1:2], format: "2010-01-01" "1979-10-14"
```

These functions are useful in their own right, but are also an important building block for **readr**. Once you've learned how the individual parsers work in this section, we'll circle back and see how they fit together to parse a complete file in the next section.

Like all functions in the tidyverse, the `parse_*()` functions are uniform; the first argument is a character vector to parse, and the `na` argument specifies which strings should be treated as missing:

```
parse_integer(c("1", "231", ".", "456"), na = ".")
#> [1]   1 231  NA 456
```

If parsing fails, you'll get a warning:

```
x <- parse_integer(c("123", "345", "abc", "123.45"))
#> Warning: 2 parsing failures.
#> row col               expected actual
#>   3  -- an integer               abc
#>   4  -- no trailing characters    .45
```

And the failures will be missing in the output:

```
x
#> [1] 123 345  NA  NA
#> attr(,"problems")
#> # A tibble: 2 × 4
#>     row   col               expected actual
#>   <int> <int>                  <chr>  <chr>
#> 1     3    NA             an integer    abc
#> 2     4    NA no trailing characters    .45
```

If there are many parsing failures, you'll need to use `problems()` to get the complete set. This returns a tibble, which you can then manipulate with **dplyr**:

```
problems(x)
#> # A tibble: 2 × 4
#>     row   col               expected actual
#>   <int> <int>                  <chr>  <chr>
#> 1     3    NA             an integer    abc
#> 2     4    NA no trailing characters    .45
```

Using parsers is mostly a matter of understanding what's available and how they deal with different types of input. There are eight particularly important parsers:

- `parse_logical()` and `parse_integer()` parse logicals and integers, respectively. There's basically nothing that can go wrong with these parsers so I won't describe them here further.
- `parse_double()` is a strict numeric parser, and `parse_number()` is a flexible numeric parser. These are more complicated than you might expect because different parts of the world write numbers in different ways.
- `parse_character()` seems so simple that it shouldn't be necessary. But one complication makes it quite important: character encodings.

- `parse_factor()` creates factors, the data structure that R uses to represent categorical variables with fixed and known values.
- `parse_datetime()`, `parse_date()`, and `parse_time()` allow you to parse various date and time specifications. These are the most complicated because there are so many different ways of writing dates.

The following sections describe these parsers in more detail.

Numbers

It seems like it should be straightforward to parse a number, but three problems make it tricky:

- People write numbers differently in different parts of the world. For example, some countries use `.` in between the integer and fractional parts of a real number, while others use `,`.
- Numbers are often surrounded by other characters that provide some context, like "$1000" or "10%".
- Numbers often contain "grouping" characters to make them easier to read, like "1,000,000", and these grouping characters vary around the world.

To address the first problem, **readr** has the notion of a "locale," an object that specifies parsing options that differ from place to place. When parsing numbers, the most important option is the character you use for the decimal mark. You can override the default value of `.` by creating a new locale and setting the `decimal_mark` argument:

```
parse_double("1.23")
#> [1] 1.23
parse_double("1,23", locale = locale(decimal_mark = ","))
#> [1] 1.23
```

readr's default locale is US-centric, because generally R is US-centric (i.e., the documentation of base R is written in American English). An alternative approach would be to try and guess the defaults from your operating system. This is hard to do well, and, more importantly, makes your code fragile: even if it works on your computer, it might fail when you email it to a colleague in another country.

`parse_number()` addresses the second problem: it ignores non-numeric characters before and after the number. This is particularly useful for currencies and percentages, but also works to extract numbers embedded in text:

```
parse_number("$100")
#> [1] 100
parse_number("20%")
#> [1] 20
parse_number("It cost $123.45")
#> [1] 123
```

The final problem is addressed by the combination of `parse_number()` and the locale as `parse_number()` will ignore the "grouping mark":

```
# Used in America
parse_number("$123,456,789")
#> [1] 1.23e+08

# Used in many parts of Europe
parse_number(
  "123.456.789",
  locale = locale(grouping_mark = ".")
)
#> [1] 1.23e+08

# Used in Switzerland
parse_number(
  "123'456'789",
  locale = locale(grouping_mark = "'")
)
#> [1] 1.23e+08
```

Strings

It seems like `parse_character()` should be really simple—it could just return its input. Unfortunately life isn't so simple, as there are multiple ways to represent the same string. To understand what's going on, we need to dive into the details of how computers represent strings. In R, we can get at the underlying representation of a string using `charToRaw()`:

```
charToRaw("Hadley")
#> [1] 48 61 64 6c 65 79
```

Each hexadecimal number represents a byte of information: `48` is H, `61` is a, and so on. The mapping from hexadecimal number to character is called the encoding, and in this case the encoding is called

ASCII. ASCII does a great job of representing English characters, because it's the *American* Standard Code for Information Interchange.

Things get more complicated for languages other than English. In the early days of computing there were many competing standards for encoding non-English characters, and to correctly interpret a string you needed to know both the values and the encoding. For example, two common encodings are Latin1 (aka ISO-8859-1, used for Western European languages) and Latin2 (aka ISO-8859-2, used for Eastern European languages). In Latin1, the byte `b1` is "±", but in Latin2, it's "ą"! Fortunately, today there is one standard that is supported almost everywhere: UTF-8. UTF-8 can encode just about every character used by humans today, as well as many extra symbols (like emoji!).

readr uses UTF-8 everywhere: it assumes your data is UTF-8 encoded when you read it, and always uses it when writing. This is a good default, but will fail for data produced by older systems that don't understand UTF-8. If this happens to you, your strings will look weird when you print them. Sometimes just one or two characters might be messed up; other times you'll get complete gibberish. For example:

```
x1 <- "El Ni\xf1o was particularly bad this year"
x2 <- "\x82\xb1\x82\xf1\x82\xc9\x82\xbf\x82\xcd"
```

To fix the problem you need to specify the encoding in `parse_character()`:

```
parse_character(x1, locale = locale(encoding = "Latin1"))
#> [1] "El Niño was particularly bad this year"
parse_character(x2, locale = locale(encoding = "Shift-JIS"))
#> [1] "こんにちは"
```

How do you find the correct encoding? If you're lucky, it'll be included somewhere in the data documentation. Unfortunately, that's rarely the case, so **readr** provides `guess_encoding()` to help you figure it out. It's not foolproof, and it works better when you have lots of text (unlike here), but it's a reasonable place to start. Expect to try a few different encodings before you find the right one:

```
guess_encoding(charToRaw(x1))
#>     encoding confidence
#> 1 ISO-8859-1       0.46
#> 2 ISO-8859-9       0.23
guess_encoding(charToRaw(x2))
```

```
#>   encoding confidence
#> 1   KOI8-R       0.42
```

The first argument to `guess_encoding()` can either be a path to a file, or, as in this case, a raw vector (useful if the strings are already in R).

Encodings are a rich and complex topic, and I've only scratched the surface here. If you'd like to learn more I'd recommend reading the detailed explanation at *http://kunststube.net/encoding/*.

Factors

R uses factors to represent categorical variables that have a known set of possible values. Give `parse_factor()` a vector of known `levels` to generate a warning whenever an unexpected value is present:

```
fruit <- c("apple", "banana")
parse_factor(c("apple", "banana", "bananana"), levels = fruit)
#> Warning: 1 parsing failure.
#> row col           expected   actual
#>   3  -- value in level set bananana
#> [1] apple  banana <NA>
#> attr(,"problems")
#> # A tibble: 1 × 4
#>     row   col           expected   actual
#>   <int> <int>              <chr>    <chr>
#> 1     3    NA value in level set bananana
#> Levels: apple banana
```

But if you have many problematic entries, it's often easier to leave them as character vectors and then use the tools you'll learn about in Chapter 11 and Chapter 12 to clean them up.

Dates, Date-Times, and Times

You pick between three parsers depending on whether you want a date (the number of days since 1970-01-01), a date-time (the number of seconds since midnight 1970-01-01), or a time (the number of seconds since midnight). When called without any additional arguments:

- `parse_datetime()` expects an ISO8601 date-time. ISO8601 is an international standard in which the components of a date are organized from biggest to smallest: year, month, day, hour, minute, second:

```
parse_datetime("2010-10-01T2010")
#> [1] "2010-10-01 20:10:00 UTC"

# If time is omitted, it will be set to midnight
parse_datetime("20101010")
#> [1] "2010-10-10 UTC"
```

This is the most important date/time standard, and if you work with dates and times frequently, I recommend reading *https://en.wikipedia.org/wiki/ISO_8601*.

- `parse_date()` expects a four-digit year, a `-` or `/`, the month, a `-` or `/`, then the day:

```
parse_date("2010-10-01")
#> [1] "2010-10-01"
```

- `parse_time()` expects the hour, `:`, minutes, optionally `:` and seconds, and an optional a.m./p.m. specifier:

```
library(hms)
parse_time("01:10 am")
#> 01:10:00
parse_time("20:10:01")
#> 20:10:01
```

Base R doesn't have a great built-in class for time data, so we use the one provided in the **hms** package.

If these defaults don't work for your data you can supply your own date-time `format`, built up of the following pieces:

Year

`%Y` (4 digits).

`%y` (2 digits; 00-69 → 2000-2069, 70-99 → 1970-1999).

Month

`%m` (2 digits).

`%b` (abbreviated name, like "Jan").

`%B` (full name, "January").

Day

`%d` (2 digits).

`%e` (optional leading space).

Time

`%H` (0-23 hour format).

`%I` (0-12, must be used with `%p`).

`%p` (a.m./p.m. indicator).

`%M` (minutes).

`%S` (integer seconds).

`%OS` (real seconds).

`%Z` (time zone [a name, e.g., `America/Chicago`]). Note: beware of abbreviations. If you're American, note that "EST" is a Canadian time zone that does not have daylight saving time. It is Eastern Standard Time! We'll come back to this in "Time Zones" on page 254.

`%z` (as offset from UTC, e.g., `+0800`).

Nondigits

`%.` (skips one nondigit character).

`%*` (skips any number of nondigits).

The best way to figure out the correct format is to create a few examples in a character vector, and test with one of the parsing functions. For example:

```
parse_date("01/02/15", "%m/%d/%y")
#> [1] "2015-01-02"
parse_date("01/02/15", "%d/%m/%y")
#> [1] "2015-02-01"
parse_date("01/02/15", "%y/%m/%d")
#> [1] "2001-02-15"
```

If you're using `%b` or `%B` with non-English month names, you'll need to set the `lang` argument to `locale()`. See the list of built-in languages in `date_names_langs()`, or if your language is not already included, create your own with `date_names()`:

```
parse_date("1 janvier 2015", "%d %B %Y", locale = locale("fr"))
#> [1] "2015-01-01"
```

Exercises

1. What are the most important arguments to `locale()`?

2. What happens if you try and set `decimal_mark` and `grouping_mark` to the same character? What happens to the default value of `grouping_mark` when you set `decimal_mark` to ",“? What happens to the default value of `decimal_mark` when you set the `grouping_mark` to ".“?
3. I didn't discuss the `date_format` and `time_format` options to `locale()`. What do they do? Construct an example that shows when they might be useful.
4. If you live outside the US, create a new locale object that encapsulates the settings for the types of files you read most commonly.
5. What's the difference between `read_csv()` and `read_csv2()`?
6. What are the most common encodings used in Europe? What are the most common encodings used in Asia? Do some googling to find out.
7. Generate the correct format string to parse each of the following dates and times:

```
d1 <- "January 1, 2010"
d2 <- "2015-Mar-07"
d3 <- "06-Jun-2017"
d4 <- c("August 19 (2015)", "July 1 (2015)")
d5 <- "12/30/14" # Dec 30, 2014
t1 <- "1705"
t2 <- "11:15:10.12 PM"
```

Parsing a File

Now that you've learned how to parse an individual vector, it's time to return to the beginning and explore how **readr** parses a file. There are two new things that you'll learn about in this section:

- How **readr** automatically guesses the type of each column.
- How to override the default specification.

Strategy

readr uses a heuristic to figure out the type of each column: it reads the first 1000 rows and uses some (moderately conservative) heuristics to figure out the type of each column. You can emulate this pro-

cess with a character vector using `guess_parser()`, which returns **readr**'s best guess, and `parse_guess()`, which uses that guess to parse the column:

```
guess_parser("2010-10-01")
#> [1] "date"
guess_parser("15:01")
#> [1] "time"
guess_parser(c("TRUE", "FALSE"))
#> [1] "logical"
guess_parser(c("1", "5", "9"))
#> [1] "integer"
guess_parser(c("12,352,561"))
#> [1] "number"

str(parse_guess("2010-10-10"))
#>  Date[1:1], format: "2010-10-10"
```

The heuristic tries each of the following types, stopping when it finds a match:

logical
: Contains only "F", "T", "FALSE", or "TRUE".

integer
: Contains only numeric characters (and `-`).

double
: Contains only valid doubles (including numbers like `4.5e-5`).

number
: Contains valid doubles with the grouping mark inside.

time
: Matches the default `time_format`.

date
: Matches the default `date_format`.

date-time
: Any ISO8601 date.

If none of these rules apply, then the column will stay as a vector of strings.

Problems

These defaults don't always work for larger files. There are two basic problems:

- The first thousand rows might be a special case, and **readr** guesses a type that is not sufficiently general. For example, you might have a column of doubles that only contains integers in the first 1000 rows.
- The column might contain a lot of missing values. If the first 1000 rows contain only `NA`s, **readr** will guess that it's a character vector, whereas you probably want to parse it as something more specific.

readr contains a challenging CSV that illustrates both of these problems:

```
challenge <- read_csv(readr_example("challenge.csv"))
#> Parsed with column specification:
#> cols(
#>   x = col_integer(),
#>   y = col_character()
#> )
#> Warning: 1000 parsing failures.
#>  row col               expected             actual
#> 1001   x no trailing characters .23837975086644292
#> 1002   x no trailing characters .41167997173033655
#> 1003   x no trailing characters .7460716762579978
#> 1004   x no trailing characters .723450553836301
#> 1005   x no trailing characters .614524137461558
#> .... ... ..................... ...................
#> See problems(...) for more details.
```

(Note the use of `readr_example()`, which finds the path to one of the files included with the package.)

There are two printed outputs: the column specification generated by looking at the first 1000 rows, and the first five parsing failures. It's always a good idea to explicitly pull out the `problems()`, so you can explore them in more depth:

```
problems(challenge)
#> # A tibble: 1,000 × 4
#>     row   col               expected             actual
#>   <int> <chr>                  <chr>              <chr>
#> 1  1001     x no trailing characters .23837975086644292
#> 2  1002     x no trailing characters .41167997173033655
#> 3  1003     x no trailing characters  .7460716762579978
```

```
#> 4  1004     x no trailing characters   .723450553836301
#> 5  1005     x no trailing characters   .614524137461558
#> 6  1006     x no trailing characters   .473980569280684
#> # ... with 994 more rows
```

A good strategy is to work column by column until there are no problems remaining. Here we can see that there are a lot of parsing problems with the `x` column—there are trailing characters after the integer value. That suggests we need to use a double parser instead.

To fix the call, start by copying and pasting the column specification into your original call:

```
challenge <- read_csv(
  readr_example("challenge.csv"),
  col_types = cols(
    x = col_integer(),
    y = col_character()
  )
)
```

Then you can tweak the type of the `x` column:

```
challenge <- read_csv(
  readr_example("challenge.csv"),
  col_types = cols(
    x = col_double(),
    y = col_character()
  )
)
```

That fixes the first problem, but if we look at the last few rows, you'll see that they're dates stored in a character vector:

```
tail(challenge)
#> # A tibble: 6 × 2
#>       x          y
#>   <dbl>      <chr>
#> 1 0.805 2019-11-21
#> 2 0.164 2018-03-29
#> 3 0.472 2014-08-04
#> 4 0.718 2015-08-16
#> 5 0.270 2020-02-04
#> 6 0.608 2019-01-06
```

You can fix that by specifying that `y` is a date column:

```
challenge <- read_csv(
  readr_example("challenge.csv"),
  col_types = cols(
    x = col_double(),
    y = col_date()
```

```
  )
)
tail(challenge)
#> # A tibble: 6 × 2
#>       x          y
#>   <dbl>     <date>
#> 1 0.805 2019-11-21
#> 2 0.164 2018-03-29
#> 3 0.472 2014-08-04
#> 4 0.718 2015-08-16
#> 5 0.270 2020-02-04
#> 6 0.608 2019-01-06
```

Every `parse_xyz()` function has a corresponding `col_xyz()` function. You use `parse_xyz()` when the data is in a character vector in R already; you use `col_xyz()` when you want to tell **readr** how to load the data.

I highly recommend always supplying `col_types`, building up from the printout provided by **readr**. This ensures that you have a consistent and reproducible data import script. If you rely on the default guesses and your data changes, **readr** will continue to read it in. If you want to be really strict, use `stop_for_problems()`: that will throw an error and stop your script if there are any parsing problems.

Other Strategies

There are a few other general strategies to help you parse files:

- In the previous example, we just got unlucky: if we look at just one more row than the default, we can correctly parse in one shot:

```
challenge2 <- read_csv(
                readr_example("challenge.csv"),
                guess_max = 1001
              )
#> Parsed with column specification:
#> cols(
#>   x = col_double(),
#>   y = col_date(format = "")
#> )
challenge2
#> # A tibble: 2,000 × 2
#>       x      y
#>   <dbl> <date>
#> 1   404   <NA>
```

```
#> 2  4172   <NA>
#> 3  3004   <NA>
#> 4   787   <NA>
#> 5    37   <NA>
#> 6  2332   <NA>
#> # ... with 1,994 more rows
```

- Sometimes it's easier to diagnose problems if you just read in all the columns as character vectors:

```
challenge2 <- read_csv(readr_example("challenge.csv"),
  col_types = cols(.default = col_character())
)
```

  This is particularly useful in conjunction with `type_convert()`, which applies the parsing heuristics to the character columns in a data frame:

```
df <- tribble(
  ~x,  ~y,
  "1", "1.21",
  "2", "2.32",
  "3", "4.56"
)
df
#> # A tibble: 3 × 2
#>       x     y
#>   <chr> <chr>
#> 1     1  1.21
#> 2     2  2.32
#> 3     3  4.56

# Note the column types
type_convert(df)
#> Parsed with column specification:
#> cols(
#>   x = col_integer(),
#>   y = col_double()
#> )
#> # A tibble: 3 × 2
#>       x     y
#>   <int> <dbl>
#> 1     1  1.21
#> 2     2  2.32
#> 3     3  4.56
```

- If you're reading a very large file, you might want to set `n_max` to a smallish number like 10,000 or 100,000. That will accelerate your iterations while you eliminate common problems.

- If you're having major parsing problems, sometimes it's easier to just read into a character vector of lines with `read_lines()`, or even a character vector of length 1 with `read_file()`. Then you can use the string parsing skills you'll learn later to parse more exotic formats.

Writing to a File

readr also comes with two useful functions for writing data back to disk: `write_csv()` and `write_tsv()`. Both functions increase the chances of the output file being read back in correctly by:

- Always encoding strings in UTF-8.
- Saving dates and date-times in ISO8601 format so they are easily parsed elsewhere.

If you want to export a CSV file to Excel, use `write_excel_csv()`—this writes a special character (a "byte order mark") at the start of the file, which tells Excel that you're using the UTF-8 encoding.

The most important arguments are `x` (the data frame to save) and `path` (the location to save it). You can also specify how missing values are written with `na`, and if you want to `append` to an existing file:

```
write_csv(challenge, "challenge.csv")
```

Note that the type information is lost when you save to CSV:

```
challenge
#> # A tibble: 2,000 × 2
#>       x      y
#>   <dbl> <date>
#> 1   404   <NA>
#> 2  4172   <NA>
#> 3  3004   <NA>
#> 4   787   <NA>
#> 5    37   <NA>
#> 6  2332   <NA>
#> # ... with 1,994 more rows
write_csv(challenge, "challenge-2.csv")
read_csv("challenge-2.csv")
#> Parsed with column specification:
#> cols(
#>   x = col_double(),
#>   y = col_character()
#> )
```

```
#> # A tibble: 2,000 × 2
#>       x     y
#>   <dbl> <chr>
#> 1   404  <NA>
#> 2  4172  <NA>
#> 3  3004  <NA>
#> 4   787  <NA>
#> 5    37  <NA>
#> 6  2332  <NA>
#> # ... with 1,994 more rows
```

This makes CSVs a little unreliable for caching interim results—you need to re-create the column specification every time you load in. There are two alternatives:

- `write_rds()` and `read_rds()` are uniform wrappers around the base functions `readRDS()` and `saveRDS()`. These store data in R's custom binary format called RDS:

```
write_rds(challenge, "challenge.rds")
read_rds("challenge.rds")
#> # A tibble: 2,000 × 2
#>       x      y
#>   <dbl> <date>
#> 1   404   <NA>
#> 2  4172   <NA>
#> 3  3004   <NA>
#> 4   787   <NA>
#> 5    37   <NA>
#> 6  2332   <NA>
#> # ... with 1,994 more rows
```

- The **feather** package implements a fast binary file format that can be shared across programming languages:

```
library(feather)
write_feather(challenge, "challenge.feather")
read_feather("challenge.feather")
#> # A tibble: 2,000 x 2
#>       x      y
#>   <dbl> <date>
#> 1   404   <NA>
#> 2  4172   <NA>
#> 3  3004   <NA>
#> 4   787   <NA>
#> 5    37   <NA>
#> 6  2332   <NA>
#> # ... with 1,994 more rows
```

feather tends to be faster than RDS and is usable outside of R. RDS supports list-columns (which you'll learn about in Chapter 20); **feather** currently does not.

Other Types of Data

To get other types of data into R, we recommend starting with the tidyverse packages listed next. They're certainly not perfect, but they are a good place to start. For rectangular data:

- **haven** reads SPSS, Stata, and SAS files.
- **readxl** reads Excel files (both *.xls* and *.xlsx*).
- **DBI**, along with a database-specific backend (e.g., **RMySQL**, **RSQLite**, **RPostgreSQL**, etc.) allows you to run SQL queries against a database and return a data frame.

For hierarchical data: use **jsonlite** (by Jeroen Ooms) for JSON, and **xml2** for XML. Jenny Bryan has some excellent worked examples at *https://jennybc.github.io/purrr-tutorial/*.

For other file types, try the R data import/export manual and the **rio** package.

CHAPTER 9

Tidy Data with tidyr

Introduction

> Happy families are all alike; every unhappy family is unhappy in its own way.
>
> —Leo Tolstoy

> Tidy datasets are all alike, but every messy dataset is messy in its own way.
>
> —Hadley Wickham

In this chapter, you will learn a consistent way to organize your data in R, an organization called *tidy data*. Getting your data into this format requires some up-front work, but that work pays off in the long term. Once you have tidy data and the tidy tools provided by packages in the tidyverse, you will spend much less time munging data from one representation to another, allowing you to spend more time on the analytic questions at hand.

This chapter will give you a practical introduction to tidy data and the accompanying tools in the **tidyr** package. If you'd like to learn more about the underlying theory, you might enjoy the *Tidy Data* paper published in the *Journal of Statistical Software*.

Prerequisites

In this chapter we'll focus on **tidyr**, a package that provides a bunch of tools to help tidy up your messy datasets. **tidyr** is a member of the core tidyverse.

```
library(tidyverse)
```

Tidy Data

You can represent the same underlying data in multiple ways. The following example shows the same data organized in four different ways. Each dataset shows the same values of four variables, *country*, *year*, *population*, and *cases*, but each dataset organizes the values in a different way:

```
table1
#> # A tibble: 6 × 4
#>       country  year  cases population
#>         <chr> <int>  <int>      <int>
#> 1 Afghanistan  1999    745   19987071
#> 2 Afghanistan  2000   2666   20595360
#> 3      Brazil  1999  37737  172006362
#> 4      Brazil  2000  80488  174504898
#> 5       China  1999 212258 1272915272
#> 6       China  2000 213766 1280428583
table2
#> # A tibble: 12 × 4
#>       country  year       type     count
#>         <chr> <int>      <chr>     <int>
#> 1 Afghanistan  1999      cases       745
#> 2 Afghanistan  1999 population  19987071
#> 3 Afghanistan  2000      cases      2666
#> 4 Afghanistan  2000 population  20595360
#> 5      Brazil  1999      cases     37737
#> 6      Brazil  1999 population 172006362
#> # ... with 6 more rows
table3
#> # A tibble: 6 × 3
#>       country  year              rate
#> *       <chr> <int>             <chr>
#> 1 Afghanistan  1999      745/19987071
#> 2 Afghanistan  2000     2666/20595360
#> 3      Brazil  1999   37737/172006362
#> 4      Brazil  2000   80488/174504898
#> 5       China  1999 212258/1272915272
#> 6       China  2000 213766/1280428583

# Spread across two tibbles
table4a  # cases
#> # A tibble: 3 × 3
#>       country `1999` `2000`
#> *       <chr>  <int>  <int>
#> 1 Afghanistan    745   2666
#> 2      Brazil  37737  80488
#> 3       China 212258 213766
```

```
table4b  # population
#> # A tibble: 3 × 3
#>       country     `1999`     `2000`
#> *       <chr>      <int>      <int>
#> 1 Afghanistan   19987071   20595360
#> 2      Brazil  172006362  174504898
#> 3       China 1272915272 1280428583
```

These are all representations of the same underlying data, but they are not equally easy to use. One dataset, the tidy dataset, will be much easier to work with inside the tidyverse.

There are three interrelated rules which make a dataset tidy:

1. Each variable must have its own column.
2. Each observation must have its own row.
3. Each value must have its own cell.

Figure 9-1 shows the rules visually.

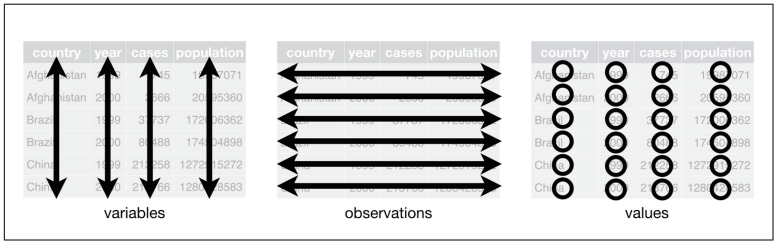


Figure 9-1. The following three rules make a dataset tidy: variables are in columns, observations are in rows, and values are in cells

These three rules are interrelated because it's impossible to only satisfy two of the three. That interrelationship leads to an even simpler set of practical instructions:

1. Put each dataset in a tibble.
2. Put each variable in a column.

In this example, only `table1` is tidy. It's the only representation where each column is a variable.

Why ensure that your data is tidy? There are two main advantages:

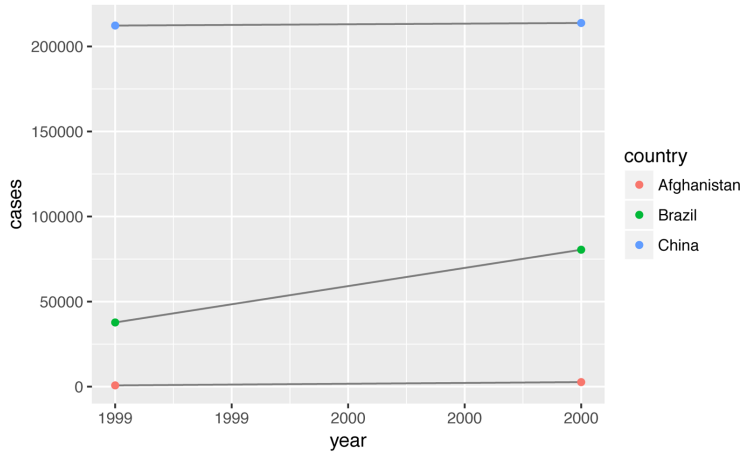
- There's a general advantage to picking one consistent way of storing data. If you have a consistent data structure, it's easier to

learn the tools that work with it because they have an underlying uniformity.

- There's a specific advantage to placing variables in columns because it allows R's vectorized nature to shine. As you learned in "Useful Creation Functions" on page 56 and "Useful Summary Functions" on page 66, most built-in R functions work with vectors of values. That makes transforming tidy data feel particularly natural.

dplyr, **ggplot2**, and all the other packages in the tidyverse are designed to work with tidy data. Here are a couple of small examples showing how you might work with `table1`:

```
# Compute rate per 10,000
table1 %>%
  mutate(rate = cases / population * 10000)
#> # A tibble: 6 × 5
#>       country  year  cases population  rate
#>         <chr> <int>  <int>      <int> <dbl>
#> 1 Afghanistan  1999    745   19987071 0.373
#> 2 Afghanistan  2000   2666   20595360 1.294
#> 3      Brazil  1999  37737  172006362 2.194
#> 4      Brazil  2000  80488  174504898 4.612
#> 5       China  1999 212258 1272915272 1.667
#> 6       China  2000 213766 1280428583 1.669

# Compute cases per year
table1 %>%
  count(year, wt = cases)
#> # A tibble: 2 × 2
#>    year      n
#>   <int>  <int>
#> 1  1999 250740
#> 2  2000 296920

# Visualize changes over time
library(ggplot2)
ggplot(table1, aes(year, cases)) +
  geom_line(aes(group = country), color = "grey50") +
  geom_point(aes(color = country))
```



Exercises

1. Using prose, describe how the variables and observations are organized in each of the sample tables.
2. Compute the `rate` for `table2`, and `table4a` + `table4b`. You will need to perform four operations:
   a. Extract the number of TB cases per country per year.
   b. Extract the matching population per country per year.
   c. Divide cases by population, and multiply by 10,000.
   d. Store back in the appropriate place.

   Which representation is easiest to work with? Which is hardest? Why?
3. Re-create the plot showing change in cases over time using `table2` instead of `table1`. What do you need to do first?

Spreading and Gathering

The principles of tidy data seem so obvious that you might wonder if you'll ever encounter a dataset that isn't tidy. Unfortunately, however, most data that you will encounter will be untidy. There are two main reasons:

- Most people aren't familiar with the principles of tidy data, and it's hard to derive them yourself unless you spend a *lot* of time working with data.
- Data is often organized to facilitate some use other than analysis. For example, data is often organized to make entry as easy as possible.

This means for most real analyses, you'll need to do some tidying. The first step is always to figure out what the variables and observations are. Sometimes this is easy; other times you'll need to consult with the people who originally generated the data. The second step is to resolve one of two common problems:

- One variable might be spread across multiple columns.
- One observation might be scattered across multiple rows.

Typically a dataset will only suffer from one of these problems; it'll only suffer from both if you're really unlucky! To fix these problems, you'll need the two most important functions in **tidyr**: `gather()` and `spread()`.

Gathering

A common problem is a dataset where some of the column names are not names of variables, but *values* of a variable. Take `table4a`; the column names `1999` and `2000` represent values of the `year` variable, and each row represents two observations, not one:

```
table4a
#> # A tibble: 3 × 3
#>       country `1999` `2000`
#> *       <chr>  <int>  <int>
#> 1 Afghanistan    745   2666
#> 2      Brazil  37737  80488
#> 3       China 212258 213766
```

To tidy a dataset like this, we need to *gather* those columns into a new pair of variables. To describe that operation we need three parameters:

- The set of columns that represent values, not variables. In this example, those are the columns `1999` and `2000`.

- The name of the variable whose values form the column names. I call that the `key`, and here it is `year`.
- The name of the variable whose values are spread over the cells. I call that `value`, and here it's the number of `cases`.

Together those parameters generate the call to `gather()`:

```
table4a %>%
  gather(`1999`, `2000`, key = "year", value = "cases")
#> # A tibble: 6 × 3
#>       country  year  cases
#>         <chr> <chr>  <int>
#> 1 Afghanistan  1999    745
#> 2      Brazil  1999  37737
#> 3       China  1999 212258
#> 4 Afghanistan  2000   2666
#> 5      Brazil  2000  80488
#> 6       China  2000 213766
```

The columns to gather are specified with `dplyr::select()` style notation. Here there are only two columns, so we list them individually. Note that "1999" and "2000" are nonsyntactic names so we have to surround them in backticks. To refresh your memory of the other ways to select columns, see "Select Columns with select()" on page 51.

In the final result, the gathered columns are dropped, and we get new `key` and `value` columns. Otherwise, the relationships between the original variables are preserved. Visually, this is shown in Figure 9-2. We can use `gather()` to tidy `table4b` in a similar fashion. The only difference is the variable stored in the cell values:

```
table4b %>%
  gather(`1999`, `2000`, key = "year", value = "population")
#> # A tibble: 6 × 3
#>       country  year population
#>         <chr> <chr>      <int>
#> 1 Afghanistan  1999   19987071
#> 2      Brazil  1999  172006362
#> 3       China  1999 1272915272
#> 4 Afghanistan  2000   20595360
#> 5      Brazil  2000  174504898
#> 6       China  2000 1280428583
```

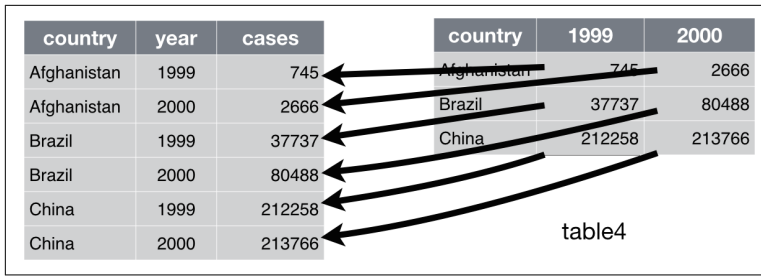


Figure 9-2. Gathering table4 into a tidy form

To combine the tidied versions of `table4a` and `table4b` into a single tibble, we need to use `dplyr::left_join()`, which you'll learn about in Chapter 10:

```
tidy4a <- table4a %>%
  gather(`1999`, `2000`, key = "year", value = "cases")
tidy4b <- table4b %>%
  gather(`1999`, `2000`, key = "year", value = "population")
left_join(tidy4a, tidy4b)
#> Joining, by = c("country", "year")
#> # A tibble: 6 × 4
#>       country  year  cases population
#>         <chr> <chr>  <int>      <int>
#> 1 Afghanistan  1999    745   19987071
#> 2      Brazil  1999  37737  172006362
#> 3       China  1999 212258 1272915272
#> 4 Afghanistan  2000   2666   20595360
#> 5      Brazil  2000  80488  174504898
#> 6       China  2000 213766 1280428583
```

Spreading

Spreading is the opposite of gathering. You use it when an observation is scattered across multiple rows. For example, take `table2`—an observation is a country in a year, but each observation is spread across two rows:

```
table2
#> # A tibble: 12 × 4
#>       country  year       type     count
#>         <chr> <int>      <chr>     <int>
#> 1 Afghanistan  1999      cases       745
#> 2 Afghanistan  1999 population  19987071
#> 3 Afghanistan  2000      cases      2666
#> 4 Afghanistan  2000 population  20595360
#> 5      Brazil  1999      cases     37737
```